

1. Turing Machines

- **Definition:** A Turing Machine (TM) is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$.
 - **Design:** Infinite tape, read/write head, finite control. **Example:** TM for $L = \{a^n b^n \mid n \geq 1\}$.
 - **Extensions:**
 - Multi-tape TM
 - Non-deterministic TM
 - Stay-put TM
 - **Church–Turing Thesis:** All algorithmically solvable problems can be solved by TM.
-

2. Recursive and Recursively Enumerable Languages

- **Recursive (Decidable):** TM halts for all inputs.
 - **Recursively Enumerable (RE):** TM halts for valid inputs; may loop on invalid ones.
 - **Closure Properties:**
 - Recursive: Union, Intersection, Complement
 - RE: Union, Intersection (not Complement)
-

3. Halting Problem

- **Definition:** Whether a TM halts on a given input.
 - **Result:** Undecidable using contradiction (diagonalization).
-

4. Rice's Theorem & Recursion Theorem

- **Rice's Theorem:** All non-trivial properties of TM languages are undecidable.
 - **Recursion Theorem:** TM can refer to its own description (used in self-replicating code/quines).
-

5. Complexity Classes

- **P:** Problems solvable in polynomial time.
 - **NP:** Problems verifiable in polynomial time.
 - **co-NP:** Complements of NP problems.
 - **PSPACE:** Polynomial space solvable problems.
 - **NPSPACE:** Equal to PSPACE (Savitch's Theorem).
-

6. Polynomial Time Reductions, NP-Completeness & NP-Hardness

- **Reduction ($\leq_p$):** Solving one problem using another in polynomial time.
- **NP-Complete:** In NP + every NP problem reduces to it.
 - Examples: SAT, 3-SAT, Clique, Vertex Cover.
- **NP-Hard:** As hard as NP-complete; not necessarily in NP.
 - Examples: Halting Problem, Tiling Problem.